A PROJECT REPORT ON,

“Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports”

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

SUBMITTED BY

Vishwajeet Tulse	**B5533**
Pravin Misal	_____
Rohan Gund	_____



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE-41

SAVITRIBAI PHULE PUNE UNIVERSITY

2026 - 27

Sinhgad Institutes

CERTIFICATE

This is to certify that the project report entitles

“Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports”

Submitted by

Vishwajeet Tulse	**BE Roll No: B5533**
Pravin Misal	**BE Roll No: _____**
Rohan Gund	**BE Roll No: _____**

is a bonafide work carried out by them under the supervision of **Prof. S. P. Bendale** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University as a part of Laboratory Practice IV work syllabus (Final year Computer Engineering).

(Prof. S. P. Bendale)	**(Prof. S. P. Bendale)**
Subject In-charge	**Head of Department**

ACKNOWLEDGEMENT

It is indeed a great pleasure and moment of immense satisfaction for us to present a project report on “Manual Testing of an E-Commerce Website Using Test Cases and Bug Reports” amongst a wide panorama that provided us inspiring guidance and encouragement, we take the opportunity to thank those who gave us the indebted assistance. We wish to extend our cordial gratitude with profound thanks to our internal guide Prof. S. P. Bendale for his everlasting guidance. It was his inspiration and encouragement which helped us in completing our project.

Our sincere thanks and deep gratitude to Head of Department, Prof. S. P. Bendale and other faculty members; but also to all those individuals involved both directly and indirectly for their help in all aspects of the project.

At last but not least we express our sincere thanks to our Institute’s Principal Dr. Shivprasad Patil for providing us infrastructure and technical environment.

Vishwajeet Tulse
Pravin Misal
Rohan Gund

ABSTRACT

Manual testing is the foundation of software quality assurance: a tester studies the requirements, designs test cases, executes them by hand in the real application and records what actually happens. In this project the complete manual testing process is applied to Swag Labs (`https://www.saucedemo.com`), a publicly available demo e-commerce website that offers login, a product catalogue, a shopping cart and a checkout flow. The site provides six test user accounts, several of which deliberately expose functional, visual and performance defects, which makes it an ideal subject for learning defect detection and reporting.

The work followed the standard test life cycle. A test plan defined the scope, strategy, environment and entry/exit criteria. Twenty test scenarios were derived from the features of the application and expanded into thirty-nine test cases written in an Excel sheet with pre-conditions, steps, test data, expected result, actual result, status and priority. Black-box techniques such as equivalence partitioning, boundary value analysis and error guessing were used to design positive and negative cases, especially for the login and checkout forms. Every test case was executed manually in Microsoft Edge on Windows 11 with the appropriate test user and evidence screenshots were captured.

Twenty-eight test cases passed and eleven failed. The failures were analysed and consolidated into nine defect reports, each with severity, priority, steps to reproduce, expected versus actual behaviour and a screenshot. The defects range from a critical checkout blocker to wrong product images, broken sorting, wrong prices, missing input validation and a cart that survives logout. A requirement traceability matrix links every requirement to its test cases. The outcome is a complete and reproducible manual testing package that demonstrates test planning, test design, execution, defect reporting and traceability.

CONTENTS

CHAPTER		TITLE	PAGE NO
1		**Problem Statement Definition**	6
2		**Software Requirements Specification**	6
	2.1	**Introduction**	6
	2.2	**Scope**	6
	2.3	**Requirements**	6
3		**System Implementation**	7
	3.1	**Technologies and Tools**	7
	3.2	**Test Design Approach**	7
4		**Working**	8
	4.1	**Test Scenarios and Test Cases**	8
	4.2	**Test Execution and Defect Reports**	9
	4.3	**Results Summary**	12
5		**Advantages and Limitations**	12
6		**Conclusion**	12

1. Problem Statement Definition

Perform manual testing of an e-commerce website (Swag Labs, `https://www.saucedemo.com`) by preparing a test plan and test scenarios, designing positive and negative test cases in an Excel sheet, executing them by hand with the available test users, recording actual results, and documenting all defects found in structured bug reports with severity, priority, steps to reproduce and evidence. Regular-expression style validation rules (e-mail, numeric postal code, name formats) are used to design the test cases for the input forms.

2. Software Requirements Specification

2.1 Introduction

Swag Labs is a web-based demo online store. A customer logs in with a username and password, sees a list of six products with name, image, description and price, sorts them by name or price, adds products to a shopping cart, reviews the cart, enters first name, last name and postal code at checkout, reviews the order summary with item total, tax and total, and finishes the order. A side menu offers All Items, About, Logout and Reset App State. The site provides six test users with the same password `secret_sauce`: `standard_user` (normal behaviour), `locked_out_user` (login refused), `problem_user` (functional bugs), `performance_glitch_user` (slow), `error_user` (errors) and `visual_user` (layout bugs).

2.2 Scope

In scope: functional, negative and UI testing of the login module, products page (listing, sorting, images, prices), cart (add, remove, badge count, persistence), checkout (form validation, totals, order completion), side menu and logout, and protected page access.

Out of scope: the payment gateway (the demo uses a fixed dummy card), load and security testing, mobile devices, and browsers other than Microsoft Edge.

2.3 Requirements

Functional requirements of the application under test:

- REQ-01: User can log in with valid credentials and is blocked otherwise.
- REQ-02: Products are listed with correct details and can be sorted.
- REQ-03: User can add and remove products in the cart.
- REQ-04: Checkout validates customer information.
- REQ-05: Order overview shows correct totals and the order can be completed.

- REQ-06: Navigation menu and logout work; protected pages need login.

Software requirements for testing:

- Windows 11 (any OS with a modern browser is sufficient)
- Microsoft Edge 142 (or Google Chrome)
- Microsoft Excel / LibreOffice Calc for the test case sheet
- Python 3 with openpyxl and selenium (only for generating the workbook and re-capturing evidence screenshots)
- Internet connection to `https://www.saucedemo.com`

3. System Implementation

3.1 Technologies and Tools

Microsoft Edge is the browser in which all test cases were executed. Its developer tools were used to inspect error messages and element states.

Microsoft Excel holds the test artifacts in one workbook, `SwagLabs_Test_Cases.xlsx`, with four sheets: Test Cases, Defect Reports, RTM (requirement traceability matrix) and Summary. Each test case row has ID, module, title, pre-condition, test steps, test data, expected result, actual result, status, priority, defect ID and evidence file.

Python with openpyxl generates the Excel workbook and the Markdown documents from a single data file, so that the sheet, the documents and this report always agree. **Selenium** was used only to re-create the evidence screenshots in a repeatable way.

Git and GitHub store all artifacts: `https://github.com/VishwajeetTulse/ecommerce-manual-testin`

3.2 Test Design Approach

Black-box techniques were applied to design the test cases:

- **Equivalence partitioning:** username $\in$ {valid, empty, unknown, locked}; password $\in$ {correct, wrong, empty}; first/last name $\in$ {letters, digits, symbols, empty}; postal code $\in$ {digits, letters, empty}. One test case per class.
- **Boundary value analysis:** empty fields and single-character inputs on the forms.
- **Error guessing:** the special test users (`problem_user`, `error_user`, `visual_user`, `performance_glitch_user`) were used to provoke failures.
- **Regular-expression rules** for the checkout form: name must match `^[A-Za-z ]+$` and postal code must match `^[0-9]{5,6}$`; test data was chosen on both sides of each rule.

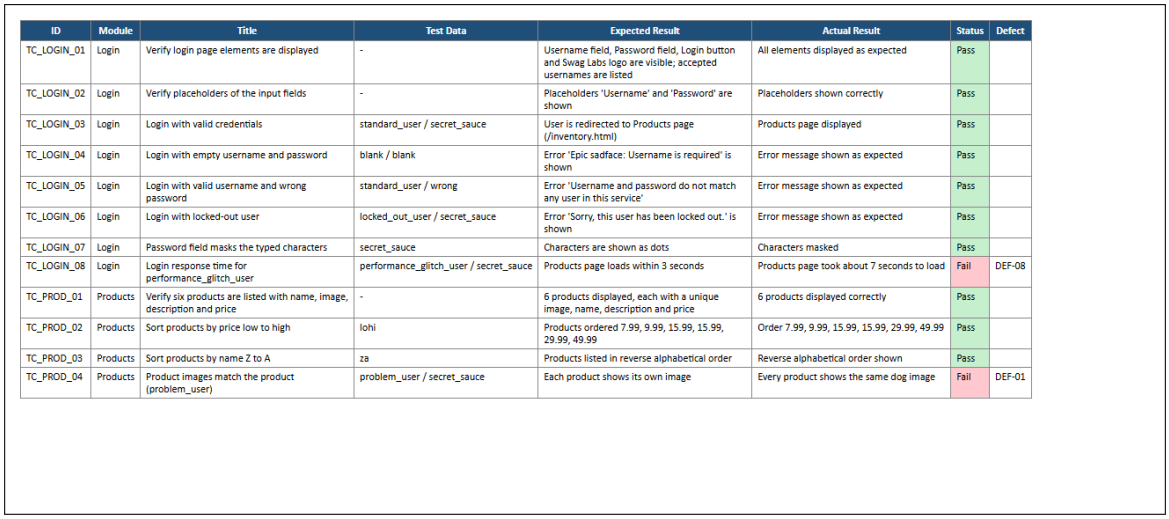
Severity scale: Critical (blocks a main flow), High (major feature wrong), Medium (works

partly or with a workaround), Low (cosmetic). Priority (High/Medium/Low) states how soon the defect should be fixed.

4. Working

4.1 Test Scenarios and Test Cases

Twenty test scenarios were written, for example: verify login with valid, empty, wrong and locked-out credentials; verify sorting by name and price; verify adding products updates the cart badge; verify mandatory field validation on the checkout form; verify overview totals and tax; verify logout and protected page access. Each scenario was expanded into one or more of the 39 test cases. The distribution is: Login 8, Products 8, Cart 6, Checkout 11, Menu 3, UI 3.



ID	Module	Title	Test Data	Expected Result	Actual Result	Status	Defect
TC_LOGIN_01	Login	Verify login page elements are displayed	-	Username field, Password field, Login button and Swag Labs logo are visible; accepted usernames are listed	All elements displayed as expected	Pass	
TC_LOGIN_02	Login	Verify placeholders of the input fields	-	Placeholders 'Username' and 'Password' are shown	Placeholders shown correctly	Pass	
TC_LOGIN_03	Login	Login with valid credentials	standard_user / secret_sauce	User is redirected to Products page (/inventory.html)	Products page displayed	Pass	
TC_LOGIN_04	Login	Login with empty username and password	blank / blank	Error 'Epic sadface: Username is required' is shown	Error message shown as expected	Pass	
TC_LOGIN_05	Login	Login with valid username and wrong password	standard_user / wrong	Error 'Username and password do not match any user in this service'	Error message shown as expected	Pass	
TC_LOGIN_06	Login	Login with locked-out user	locked_out_user / secret_sauce	Error 'Sorry, this user has been locked out.' is shown	Error message shown as expected	Pass	
TC_LOGIN_07	Login	Password field masks the typed characters	secret_sauce	Characters are shown as dots	Characters masked	Pass	
TC_LOGIN_08	Login	Login response time for performance_glitch_user	performance_glitch_user / secret_sauce	Products page loads within 3 seconds	Products page took about 7 seconds to load	Fail	DEF-08
TC_PROD_01	Products	Verify six products are listed with name, image, description and price	-	6 products displayed, each with a unique image, name, description and price	6 products displayed correctly	Pass	
TC_PROD_02	Products	Sort products by price low to high	lohi	Products ordered 7.99, 9.99, 15.99, 15.99, 29.99, 49.99	Order 7.99, 9.99, 15.99, 15.99, 29.99, 49.99	Pass	
TC_PROD_03	Products	Sort products by name Z to A	za	Products listed in reverse alphabetical order	Reverse alphabetical order shown	Pass	
TC_PROD_04	Products	Product images match the product (problem_user)	problem_user / secret_sauce	Each product shows its own image	Every product shows the same dog image	Fail	DEF-01

Figure 1: Test case sheet in Excel (first rows)

Example test case:

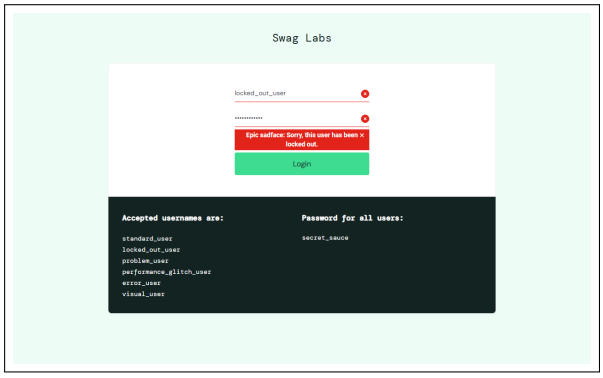
```
TC_CHK_06   Module: Checkout   Priority: Medium
Pre-condition : cart has items; on "Checkout: Your Information" page
Steps         : 1. Enter First Name "Vishwajeet"   2. Enter Last Name "Tulse"
                3. Enter Postal Code "abc"          4. Click Continue
Test data     : Vishwajeet / Tulse / abc
Expected      : validation error for Postal Code
Actual        : form accepted, Overview page displayed
Status        : Fail   ->  DEF-05
```

4.2 Test Execution and Defect Reports

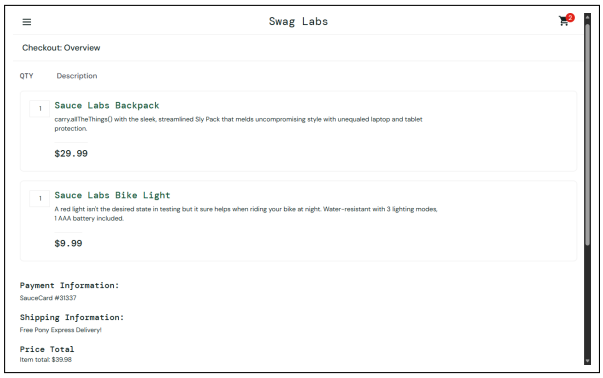
Each test case was executed by logging in as the required user, performing the steps with the given test data and comparing the observed result with the expected result. A screenshot was captured for every failure. The side-menu option Reset App State was used between cases that need an empty cart.



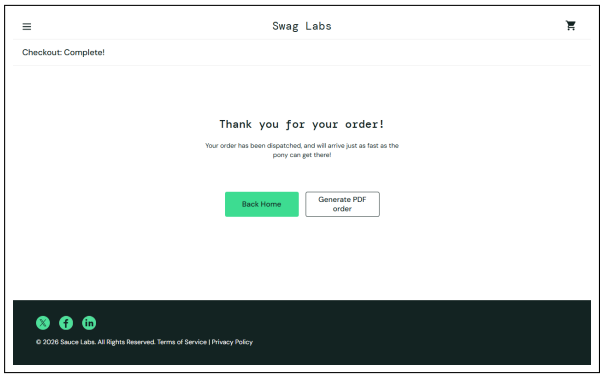
(a) Login page (TC_LOGIN_01)



(b) Locked-out user error (TC_LOGIN_06)



(c) Checkout overview (TC_CHK_07)

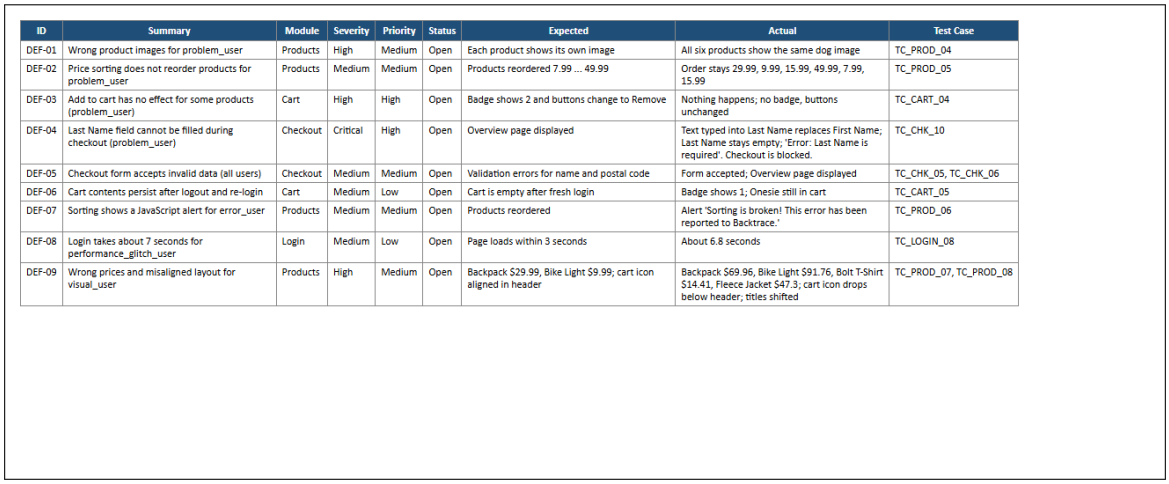


(d) Order complete (TC_CHK_09)

Figure 2: Evidence of passed test cases

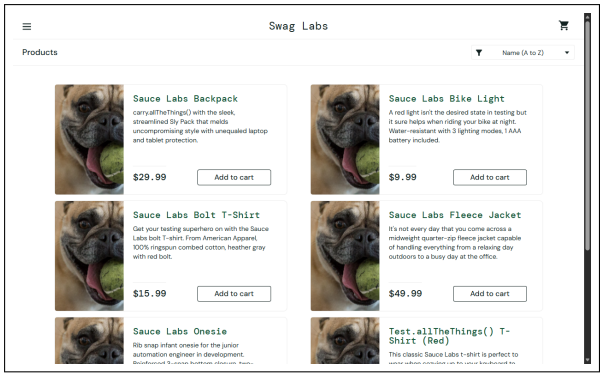
Eleven test cases failed and were consolidated into nine defect reports:

ID	Summary	Severity	Priority
DEF-01	Wrong product images for problem_user (same image on all products)	High	Medium
DEF-02	Price sorting does not reorder products for problem_user	Medium	Medium
DEF-03	Add to cart has no effect for some products (problem_user)	High	High
DEF-04	Last Name field cannot be filled during checkout (problem_user); checkout blocked	Critical	High
DEF-05	Checkout form accepts invalid data: numeric first name, symbols in last name, letters in postal code (all users)	Medium	Medium
DEF-06	Cart contents persist after logout and re-login	Medium	Low
DEF-07	Sorting shows a JavaScript alert “Sorting is broken!” for error_user	Medium	Medium
DEF-08	Login takes about 7 seconds for performance_glitch_user	Medium	Low
DEF-09	Wrong prices and misaligned layout for visual_user	High	Medium

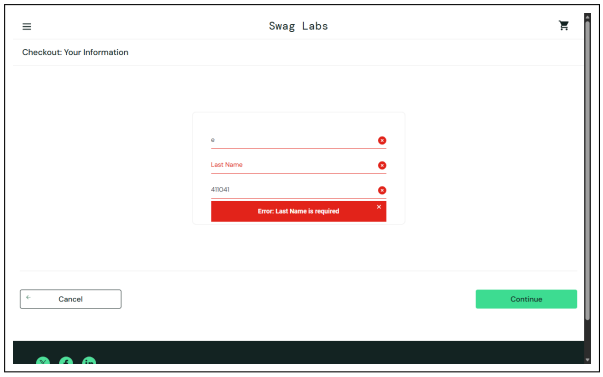


ID	Summary	Module	Severity	Priority	Status	Expected	Actual	Test Case
DEF-01	Wrong product images for problem_user	Products	High	Medium	Open	Each product shows its own image	All six products show the same dog image	TC_PROD_04
DEF-02	Price sorting does not reorder products for problem_user	Products	Medium	Medium	Open	Products reordered 7.99 ... 49.99	Order stays 29.99, 9.99, 15.99, 49.99, 7.99, 15.99	TC_PROD_05
DEF-03	Add to cart has no effect for some products (problem_user)	Cart	High	High	Open	Badge shows 2 and buttons change to Remove	Nothing happens; no badge, buttons unchanged	TC_CART_04
DEF-04	Last Name field cannot be filled during checkout (problem_user)	Checkout	Critical	High	Open	Overview page displayed	Text typed into Last Name replaces First Name; Last Name stays empty; 'Error: Last Name is required'. Checkout is blocked.	TC_CHK_10
DEF-05	Checkout form accepts invalid data (all users)	Checkout	Medium	Medium	Open	Validation errors for name and postal code	Form accepted; Overview page displayed	TC_CHK_05, TC_CHK_06
DEF-06	Cart contents persist after logout and re-login	Cart	Medium	Low	Open	Cart is empty after fresh login	Badge shows 1; Onesie still in cart	TC_CART_05
DEF-07	Sorting shows a JavaScript alert for error_user	Products	Medium	Medium	Open	Products reordered	Alert 'Sorting is broken! This error has been reported to Backtrace.'	TC_PROD_06
DEF-08	Login takes about 7 seconds for performance_glitch_user	Login	Medium	Low	Open	Page loads within 3 seconds	About 6.8 seconds	TC_LOGIN_08
DEF-09	Wrong prices and misaligned layout for visual_user	Products	High	Medium	Open	Backpack $29.99, Bike Light $9.99; cart icon aligned in header	Backpack $69.96, Bike Light $91.76, Bolt T-Shirt $14.41, Fleece Jacket $47.3; cart icon drops below header; titles shifted	TC_PROD_07, TC_PROD_08

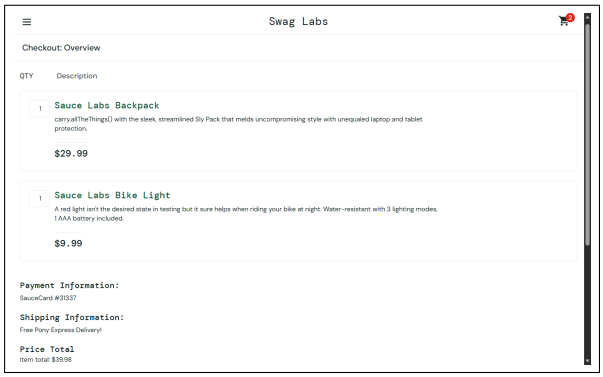
Figure 3: Defect report sheet in Excel



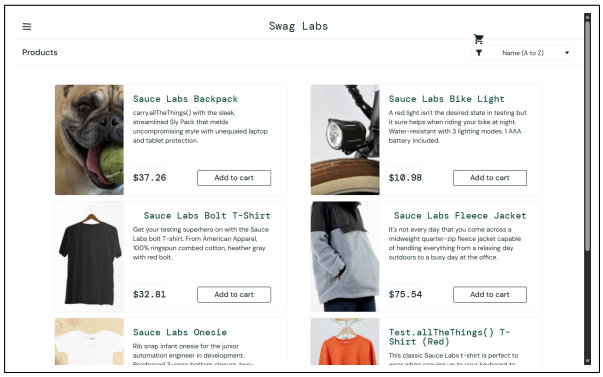
(a) DEF-01 wrong product images



(b) DEF-04 Last Name cannot be filled



(c) DEF-05 invalid data accepted



(d) DEF-09 wrong prices, misaligned cart icon

Figure 4: Evidence of defects

Example defect report:

```
Defect ID    : DEF-04
Summary      : Last Name field cannot be filled during checkout (problem_user)
Module       : Checkout        Severity: Critical      Priority: High
Environment  : Microsoft Edge 142, Windows 11, https://www.saucedemo.com
Steps        : 1. Login as problem_user, add an item, go to Checkout
               2. Type in First Name, then Last Name, then Postal Code
               3. Click Continue
Expected     : Overview page displayed
Actual       : Text typed into Last Name replaces First Name; Last Name stays
               empty; "Error: Last Name is required". Checkout is blocked.
Evidence     : screenshots/18_problem_user_lastname_bug.png
Status       : Open
```

4.3 Results Summary

Module	Test cases	Pass	Fail
Login	8	7	1
Products	8	3	5
Cart	6	4	2
Checkout	11	8	3
Menu	3	3	0
UI	3	3	0
Total	**39**	**28**	**11**

Pass percentage: 71.8%. Defects logged: 9 (1 Critical, 3 High, 5 Medium). Every requirement REQ-01 to REQ-06 is covered by at least one test case in the RTM sheet.

5. Advantages and Limitations

Advantages

- Finds real functional, visual and performance defects that an automated script would not look for.
- Needs no programming; the Excel template can be reused for any web application.
- Every failure is reproducible because steps, data and evidence are recorded.
- Requirement traceability shows exactly which features have been tested.
- Using the special test users demonstrates error guessing effectively.

Limitations

- Manual execution is slow (about two hours for 39 cases) and must be repeated after every change.
- Results depend on the tester's attention; human error is possible.
- Only Microsoft Edge on desktop was covered; no mobile or cross-browser testing.
- The application is a public demo that cannot be fixed, so the defects remain open.

6. Conclusion

This project carried out the complete manual testing process on a real e-commerce website. Starting from a test plan and twenty scenarios, thirty-nine test cases were designed with black-box techniques and executed by hand with six test users. Eleven failures were traced to nine

defects, documented with severity, priority, reproduction steps and screenshots, and a traceability matrix links every requirement to its tests. The work shows that systematic manual testing with well-chosen negative cases and special test users finds a wide range of functional, visual and performance defects. The stable passing test cases were later converted into automated Selenium scripts in the companion automation project, and the defect reports can be tracked in a tool such as Jira in future.